

CS71 PROJECT

Student Record Management System — Source Code

Data Structure: Binary Search Tree (BST)

Algorithms: Insertion, Search, Deletion and Inorder Traversal

```

# CS71 Project
# STUDENT RECORD MANAGEMENT SYSTEM
# Data Structure: Binary Search Tree (BST)
# Algorithms: BST insertion, search, deletion and inorder traversal

```

```

class StudentNode:
    def __init__(self, student_id, name, marks):
        self.student_id = student_id
        self.name = name
        self.marks = marks
        self.left = None
        self.right = None

class StudentBST:
    def __init__(self):
        self.root = None

    # INSERT STUDENT
    # Time Complexity: O(log n) average, O(n) worst case
    def insert(self, root, student_id, name, marks):
        if root is None:
            return StudentNode(student_id, name, marks)

        if student_id < root.student_id:
            root.left = self.insert(
                root.left, student_id, name, marks
            )
        elif student_id > root.student_id:
            root.right = self.insert(
                root.right, student_id, name, marks
            )
        else:
            print("Student ID already exists.")

        return root

    # SEARCH STUDENT
    # Time Complexity: O(log n) average, O(n) worst case
    def search(self, root, student_id):
        if root is None or root.student_id == student_id:
            return root

        if student_id < root.student_id:
            return self.search(root.left, student_id)
        else:
            return self.search(root.right, student_id)

    # FIND MINIMUM NODE
    def find_min(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current

    # DELETE STUDENT
    # Time Complexity: O(log n) average, O(n) worst case
    def delete(self, root, student_id):

```

```

    if root is None:
        return root

    if student_id < root.student_id:
        root.left = self.delete(root.left, student_id)

    elif student_id > root.student_id:
        root.right = self.delete(root.right, student_id)

    else:
        # Case 1: no left child
        if root.left is None:
            return root.right

        # Case 2: no right child
        elif root.right is None:
            return root.left

        # Case 3: two children
        successor = self.find_min(root.right)

        root.student_id = successor.student_id
        root.name = successor.name
        root.marks = successor.marks

        root.right = self.delete(
            root.right, successor.student_id
        )

    return root

# INORDER TRAVERSAL
# Time Complexity: O(n)
def inorder(self, root):
    if root is not None:
        self.inorder(root.left)
        print(
            "ID:", root.student_id,
            "| Name:", root.name,
            "| Marks:", root.marks
        )
        self.inorder(root.right)

def add_student(tree):
    try:
        student_id = int(input("Enter Student ID: "))
        name = input("Enter Student Name: ")
        marks = float(input("Enter Marks: "))

        tree.root = tree.insert(
            tree.root, student_id, name, marks
        )
        print("Student added successfully.")

    except ValueError:
        print("Invalid input. Please enter numbers for ID and marks.")

```

```

def search_student(tree):
    try:
        student_id = int(input("Enter Student ID to search: "))
        student = tree.search(tree.root, student_id)

        if student is None:
            print("Student not found.")
        else:
            print("\nStudent Found")
            print("-----")
            print("Student ID :", student.student_id)
            print("Name      :", student.name)
            print("Marks     :", student.marks)

    except ValueError:
        print("Invalid input. Please enter a number.")

def delete_student(tree):
    try:
        student_id = int(input("Enter Student ID to delete: "))

        student = tree.search(tree.root, student_id)

        if student is None:
            print("Student not found.")
        else:
            tree.root = tree.delete(tree.root, student_id)
            print("Student deleted successfully.")

    except ValueError:
        print("Invalid input. Please enter a number.")

def display_all_students(tree):
    print("\n===== ALL STUDENTS =====")

    if tree.root is None:
        print("No student records available.")
    else:
        tree.inorder(tree.root)

def main():
    tree = StudentBST()

    while True:
        print("\n=====")
        print("      STUDENT RECORD MANAGEMENT SYSTEM")
        print("=====")
        print("1. Add Student")
        print("2. Search Student")
        print("3. Delete Student")
        print("4. Display All Students")
        print("5. Exit")

        choice = input("Enter your choice: ")

```

```
    if choice == "1":
        add_student(tree)

    elif choice == "2":
        search_student(tree)

    elif choice == "3":
        delete_student(tree)

    elif choice == "4":
        display_all_students(tree)

    elif choice == "5":
        print("Thank you for using the system.")
        break

    else:
        print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```